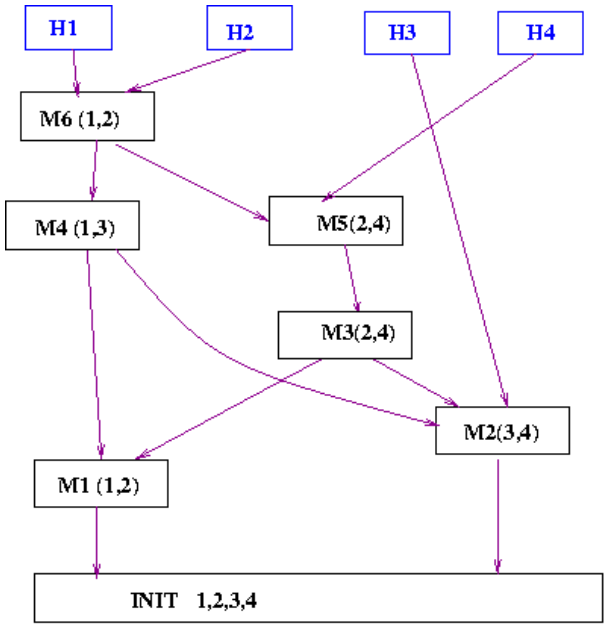
Tutorial 9

1. *The graph is an extremely useful modelling tool. Here is how a Covid tracing tool might work. Let V be the set of all persons. We say (p, q) is an edge (i) in $E1$ if their names appear on the same webpage and (ii) in $E2$ if they have been together in a common location for more than 20 minutes. What significance do the connected components in these graphs, and what does the BFS do? Does the second graph have epidemiological significance? If so, what? If not, how would you improve the graph structure to get a sharper epidemiological meaning?*

Solution: The first graph, with edges E1, gives us a measure of the closeness of two people with each other. This may be interpreted as a measure of how frequently they come in contact with each other, and clearly, the more often they come in contact, the more likely the virus is to spread from one of them to the other. Thus, we can use this as a kind of contact tracing tool; BFS gives us the shortest path between any two people, and the shorter this path is, the more likely it is that the virus has spread from one end to the other.

For every infected person, we can do a BFS over the graph, and all nodes (people) having small shortest distances from that infected person can be classified as having high risk. Any connected component thus consists of a number of people who have high chances of spreading the virus among themselves if any one gets infected. This can be used to determine the risk faced by every individual in the system, due to a particular person being infected.



The second graph, with edges $E2$, can similarly be used to identify high-risk individuals since any infected person is likely to have spread the infection to their neighbour(s) in the $E2$ graph, in the 20-minute interval where they were close together. However, this information is not sufficient for points separated by more than one edge because the graph does not contain any information about the chronology of the 20-minute interactions. For example, for two individuals separated by three edges, the virus would only spread from one end to the other if the 20-minute interactions occurred in the right order. So, we can improve this data structure by additionally maintaining a

timestamp on each edge, indicating the time at which the 20-minute interaction took place.The vertices are $V = \{[M_i, S_i] | \text{i-th meeting label}, S_i = \text{people who participated}\}$. There is an edge from $[M_i, S_i]$ to $[M_j, S_j]$ if and only if

- there is a person p common to S_i and S_j
- $i < j$
- p did not participate in any meeting between i and j.

A natural question follows. How is one to generate such a graph?

Let header(p) point to the last meeting where p participated. Suppose a new meeting M_k, S_k happens with $S_k = \{p, q, r\}$ then the code is

Algorithm 11: Insert Function

```
1  Function Insert(k, S_k):
2      New meeting m;
3      m.S ← S_k;
4      m.id ← k;
5      m.next ← [] ;                                   // Adjacency list
6      foreach p in S_k do
7          m.next ← append(m.next, header(p));
8          header(p) ← m;
9      end
10 end
```

2. *Show that a bipartite graph does not contain cycles of odd length.*

Solution: We will prove this by contradiction, let us assume there is a odd length cycle. If a graph is bipartite, we can divide the set of vertices into two sets V_1 and V_2 such that there is no edge among nodes in V_1 or among nodes in V_2. Now, let's say we color all V_1 vertices by red color and all V_2 vertices by blue color. Now, there can't be any edge between two vertices of same color otherwise the graph wasn't bipartite. This implies that nodes in the cycles should be alternatively colored red and blue. Now, relate every blue node in the cycle to a red node connected to the blue node by a edge in clockwise direction. This proves that their are equal no of blue and red nodes in the cycle. This contradicts our assumption, that the cycle is of odd length. Hence, a bipartite graph does not contain cycles of odd length.

Follow up: A graph with no odd cycles is bipartite. Is this correct? If yes, prove. If not, give a counterexample.

3. *Let us take a plane paper and draw circles and infinite lines to divide the plane into various pieces. There is an edge (p, q) between two pieces if they share a common boundary of intersection (which is more than a point). Is this graph bipartite? Under what conditions is it bipartite?*

Solution: Yes, the graph is bipartite. We'll introduce a colouring scheme for the regions created by the shapes. Follow these steps:-

1. Colour the entire plane red (or blue as you like) and create a list of **unique** line(s)/circle(s) and insert them one-by-one.

2. If there's a circle/line to be inserted, then insert it in the plane, else exit this procedure.
3. Insertion will divide some of the previously existing regions into more regions. Now flip ALL the colours present inside the circle/to the left of the line (red $\rightarrow$ blue, blue $\rightarrow$ red).
4. Goto Step 2.

Notice that this scheme ensures that no two regions that share a boundary will have the same colour. Why so? Let's assume the contrary. Everything was well till I inserted $i-1$ line(s)/circles(s) but as soon as I insert the i^{th} line/circle, I find two red (WLOG) regions that share a boundary (line/arc between two vertices (vertices include infinity too) (say B). There will be two cases:-

- B is a part of the newly inserted shape. But this case shouldn't happen as, before the flipping, both sides of B should've the same colour and hence after flipping B would've different colours across it. Now you may think why should both sides of B be of same colour before flipping, because for it to be different, B should overlap on an already existing boundary that had different colours on its sides but boundaries overlapping would only mean that the newly inserted line/circle is exactly same as one of the previously inserted ones, but I've removed that possibility (see step 1).
- B is not a part of the newly inserted shape and hence of some already existing boundary. But this shouldn't happen as otherwise something should've been noticed in the previous $i-1$ steps.

Hence such a colouring scheme that prevents two same colours from having an edge (boundary) will make the graph bipartite.

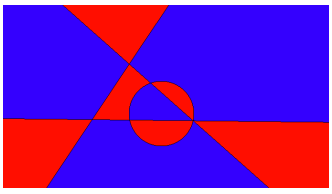


Figure 9: Initial

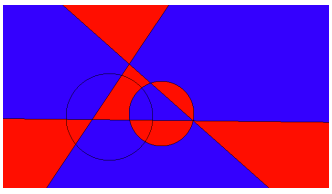


Figure 10: Insertion

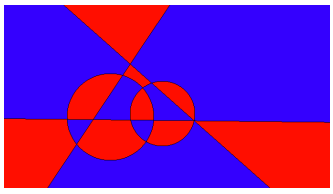
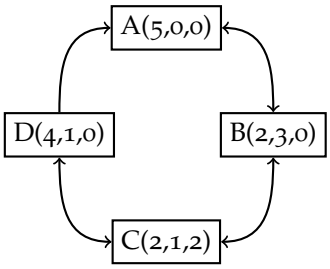


Figure 11: Flipping

4. *There are three containers, A, B, and C, with capacities of* $5, 3$, *and* 2 *liters respectively. We begin with A has* 5 *liters of milk, and B and C are empty. There are no other measuring instruments. A buyer wants* 4 *liters of milk. Can you dispense this? Model this as a graph problem with the vertex set V as the set of configurations* $c = (c1, c2, c3)$ *and an edge from c to d if d is reachable from c. Begin with* $(5, 0, 0)$. *Is this graph directed or undirected? Is it adequate to model the question: How to dispense* 4 *liters?*

Solution: At node D we can measure 4 liters. This is a directed graph because we can go directly from D to A but not reverse along that edge.

Graph below:-



5. There are many variations of BFS to solve various needs. For example, suppose that every edge $e = (u, v)$ also has a weight $w(e)$ (say the width of the road from u to v). Assume that the set of values that $w(e)$ can take is small. For a path $p = (v_1, v_2, \ldots, v_k)$, let the weight $w(p)$ be the minimum of the weights of the edges in the path. We would like to find a shortest path from a vertex s to all vertices v. If there are multiple such paths, we would like to find a path whose weight is maximum. Can we adapt BFS to detect this path?

Solution: The key changes are:

- Maintain a new variable called width[u] which records the highest width path so far.
- Whenever a new node is first encountered, both the width and distance is set.
- If an alternate path is discovered, then the width is checked and the path is updated.

See Pseudo Code in Algorithm 12. Using *parent* variable, you can trace out the path.

Algorithm 12: Shortest Path with Highest Width Path BFS

```
Data: All nodes initially have variables visited set to False, parent set to void, width_path
      set to −∞ and level set to 0
1  Function ModifiedBFS(Node s):
2      Queue Q
3      s.visited ← True
4      Q.enqueue(s)
5      while not Q.empty() do
6          curr ← Q.dequeue()
7          foreach (neighbour, edge_weight) in curr.adjacent do
8              if not neighbour.visited then
9                  neighbour.visited ← True
10                 neighbour.level ← curr.level + 1
11                 Q.enqueue(neighbour)
12             end
13             new_width_path ← min(curr.width_path, edge_weight)
14             if curr.level = neighbour.level − 1 and
                new_width_path > neighbour.width_path then
15                 neighbour.width_path ← new_width_path
16                 neighbour.parent ← curr
17             end
18         end
19     end
20 end
```

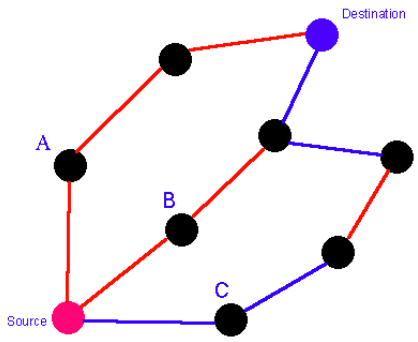
6. *Write an induction proof to show that if vertex r and v are connected in a graph G, then v will be visited in call BFSConnected(Graph G = (V, E), Vertex r, int id).*

Solution: Let's use induction on shortest distance d from r.

- Base Case: d = 1, While r is popped, all nodes from adjacency list of r is added to queue and hence get visited.
- Now, let us assume this holds for a distance k, i.e, all the nodes at a shortest distance k from a node r are visited in call BFSConnected(Graph G = (V, E), Vertex r, int id), we have to prove that all nodes at a shortest distance of k+1 will also be visited. Consider the path from r to the node, let's denote it by $r.p_0....p_k.p_{k+1}$. p_k is visited by induction hypothesis as it's at a distance of k from r and is added to BFS queue. When p_k was popped out of queue if p_{k+1} was not present in the queue then it will be added to the queue by the algorithm implying p_{k+1} will eventually get visited.

7. *Suppose that there is an undirected graph G(V,E) where the edges are colored either red or blue. Given two vertices u and v. It is desired to (i) find the shortest path irrespective of colour, (ii) find the shortest path, and of these paths, the one with the fewest red edges, (iii) a path with the fewest red edges. Draw an example where the above three paths are distinct. Clearly, to solve (i), BFS is the answer. How will you design algorithms for (ii) and (iii)?*

Solution: Example:-

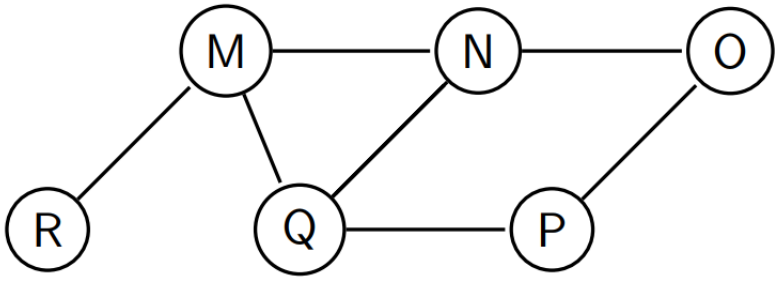


Here (i) takes A path, (ii) takes B path and (iii) takes C path.

- (i) Normal BFS will work.
- (ii) (Similar to Q5 in this Tutorial). We can modify BFS to obtain this. We want that between adjacent levels of BFS, if we have a choice between red and blue, then we choose the blue edge. Modification: if in bfs while checking in adjacency list if some node is already visited and the node is linked to it's parent with a red edge and that node's parent and current node are at same level then switch the parent of that node to the current node.
- (iii) We will do 0-1 bfs to do it. Visit all the nodes that can be visited from blue edges from source till blue edge got all exhausted. Now, visit all the nodes that can be visited from one red edge from the visited nodes. Then again visit the nodes which can be visited only with blue edge. Continue this process, till all nodes are visited. (Can you generalize it to problem with k color edges and you want to reach all the nodes with minimum no of color changes of edges?)

All the parts can be easily solved from Dijkstra's algorithm by suitable choices of weight.

8. *Is MNRQPO a possible BFS traversal for the following graph?*



Solution: No, because N was visited before Q, so O should be visited before P. This is because O will be added to queue when N was visited while P will be added when Q was visited.